



Campus Library Book Registry

Technical Documentation

Development Team

Shahd Mamdouh

Menna Sherif

Moustafa Said

Mustafa Waheed

Marwan Ashraf

Technologies of Cryptocurrencies — Final Project

Table of Contents

1. Project Overview
2. Problem Statement
3. Objectives
4. Features and Functionalities
5. Technologies Used
6. System Architecture
7. Database Design (On-Chain Data Structures)
8. API Documentation
9. Installation & Setup Guide
10. Folder Structure Explanation
11. User Roles and Permissions
12. Workflow / Business Logic
13. Screens Description (Terminal Interface)
14. Challenges and Solutions
15. Future Improvements
16. Testing Strategy
17. Deployment Process
18. Conclusion

1. Project Overview

The **Campus Library Book Registry** is a decentralized application (DApp) designed to modernize and secure library management using blockchain technology. Built as the final project for the *Technologies of Cryptocurrencies* course, this system replaces traditional centralized library databases with an immutable, tamper-proof ledger that permanently records every book loan and return on-chain.

Unlike conventional library systems that rely on mutable relational databases, this solution leverages Ethereum smart contracts (deployed locally on Ganache) to ensure that no record can be altered or deleted after the fact. Every transaction — from adding a book to returning it — is cryptographically signed and permanently stored on the blockchain.

Project Name	Campus Library Book Registry
Type	Decentralized Application (DApp)
Platform	Local Ethereum Blockchain (Ganache)
Language	Python 3 (backend/terminal) · Solidity (smart contracts)
Course	Technologies of Cryptocurrencies — Final Project
Repository	github.com/ShahdMamdouhMuhamed/-Campus-Library-Book-Registry
Team Members	Shahd Mamdouh · Menna Sherif · Moustafa Said · Mustafa Waheed · Marwan Ashraf
Token	Library Coin (LBRC) — ERC-20 Standard
Version	1.0.0 (Main Branch)

2. Problem Statement

Traditional campus library management systems suffer from significant shortcomings that undermine data integrity, transparency, and accountability. These systems typically store records in centralized databases, which are inherently vulnerable to the following issues:

- **Data Tampering:** Administrators or malicious actors can alter or delete loan records, creating disputes that cannot be resolved with certainty.
- **Lack of Transparency:** Students and faculty have no independent way to verify borrow/return history or confirm that a book is genuinely available.
- **Single Point of Failure:** A centralized server going offline or being compromised can result in total data loss.
- **No Immutable Audit Trail:** There is no cryptographically verifiable history of all actions taken in the system.
- **Manual Processes:** Many campus libraries still rely on paper records or basic spreadsheet-based tracking, which introduces human error and inefficiency.
- **No Incentive Mechanism:** Current systems lack any reward structure to encourage timely returns or responsible usage of library resources.

This project addresses all of the above by leveraging a public, immutable blockchain ledger where every state change is permanently recorded, cryptographically secured, and independently verifiable by any participant in the network.

3. Objectives

The primary objectives of the Campus Library Book Registry project are:

- **Objective 1: Build an Immutable Record System:** Ensure that every book loan and return is permanently and irreversibly recorded on the Ethereum blockchain.
- **Objective 2: Implement Role-Based Access Control:** Use Solidity's `onlyOwner` modifier to restrict sensitive operations (adding books, minting coins) exclusively to the Admin.
- **Objective 3: Issue a Custom ERC-20 Token:** Deploy Library Coin (LBRC) as a reward and utility token within the library ecosystem.
- **Objective 4: Provide a Functional Terminal Interface:** Develop a Python-based command-line application that enables both Admin and Normal User interactions with the blockchain.
- **Objective 5: Automate Administrative Analytics:** Generate on-chain analytics including dashboards, activity history, and balance snapshots without a centralized database.
- **Objective 6: Implement Emergency Controls:** Include a pause/resume mechanism to halt all user-facing operations instantly when required.
- **Objective 7: Ensure Security:** Write and execute automated tests that verify access control rules are enforced at the smart contract level.
- **Objective 8: Document and Deploy a Complete System:** Deliver a fully documented, reproducible system that demonstrates end-to-end blockchain application development.

4. Features and Functionalities

4.1 Admin (Privileged User) Features

- **Core Smart Contract Management:** Deploy and control the LibraryRegistry.sol contract. Add or update library book records.
- **Custom Coin (Library Coin):** Mint LBRC tokens and distribute them to users. Only the Admin can call the mint() function.
- **Auto-Setup Script (autosetup.py):** Deploys contracts, seeds sample books, and configures the environment automatically.
- **Access Control Module:** onlyOwner modifier protects all sensitive functions. getAdmin() provides public visibility.
- **Admin Dashboard (admin_dashboard.py):** Displays total books, total coins minted, transaction count, and the top 3 most active users.
- **Batch Operations:** Add or update multiple library books in a single transaction. Invalid entries reject the entire batch.
- **Pause & Resume (Emergency Stop):** Toggle a paused state to block all user-facing actions during incidents or maintenance.
- **Ownership Transfer:** Securely transfer admin rights to a new wallet address via transferOwnership().

4.2 Normal User Features

- **Transaction Sender:** Safely connects terminal choices to the blockchain, sending signed transactions.
- **Terminal Application (LibraryApp.py):** Main command-line interface with separate menus for users and admin.
- **User Registration & Profile:** registerUser(name) saves a display name to the user's wallet address on-chain.
- **Book Browsing:** View all registered books and their current availability status.
- **Borrow & Return Books:** Execute borrow and return transactions via signed wallet operations.
- **Personal Activity History:** Scan all blocks and display a complete log of every borrow/return for an address.
- **Coin & ETH Balance Checker:** Display LBRC token balance and ETH balance for any address.

4.3 System & Background Features

- **Security Testing (security_test.py):** Automated tests proving Normal Users cannot execute Admin functions.
- **Live Alert System (alert_monitor.py):** Background script that prints real-time alerts when books are borrowed or returned.
- **Data History Report (history_report.py):** Reads the full blockchain history and generates a report of the most borrowed books.
- **Balance Snapshot Exporter (balance_snapshot.py):** Scans all accounts and exports LBRC and ETH balances to a CSV file.
- **System Documentation:** Includes a README.md, user manual, and this technical documentation.

5. Technologies Used

Layer	Technology	Version	Purpose
Smart Contracts	Solidity	^0.8.0	Core contract logic, access control, token standard
Local Blockchain	Ganache (ganache-cli)	Latest	Local Ethereum network for development and testing
Web3 Library	web3.py	6.x	Python library to interact with Ethereum nodes
Contract Compilation	py-solc-x (solcx)	Latest	Compiles Solidity contracts from Python
Terminal Interface	Python 3	3.9+	Command-line application for all user interactions
Token Standard	ERC-20	—	Standard interface for the Library Coin (LBRC)
Testing Framework	Python unittest	Built-in	Automated security and access control tests
Version Control	Git + GitHub	—	Source code management and collaboration
Package Manager (JS)	Node.js / npm	—	Required by Ganache CLI installation
Data Export	Python csv module	Built-in	Balance snapshot export to CSV

6. System Architecture

The system follows a three-layer decentralized architecture. There is no traditional backend server or relational database — all state is stored on the blockchain.

6.1 Architecture Layers

- **Presentation Layer:** Python 3 Terminal Application (LibraryApp.py, menu.py). The user interacts entirely through a command-line interface that presents menus, reads inputs, and displays results.
- **Application / Logic Layer:** Python scripts (tx_sender.py, user_profile.py, balance_checker.py, admin_dashboard.py, etc.). These scripts translate user actions into web3.py calls and format blockchain data for display.
- **Blockchain Layer:** Ganache local Ethereum node running two Solidity smart contracts: LibraryRegistry.sol (core logic) and LibraryCoin.sol (ERC-20 token). All state changes are permanently recorded as on-chain transactions.

6.2 Communication Flow

The terminal application communicates with the Ganache node via the JSON-RPC protocol over HTTP (default: http://127.0.0.1:8545). web3.py acts as the bridge, providing a Pythonic API to:

- Compile and deploy Solidity contracts using py-solc-x
- Sign and broadcast transactions using private keys
- Call read-only (view) functions to query contract state
- Subscribe to events (Transfer, BookBorrowed, BookReturned) for live alerts
- Scan block history for analytics and reporting

6.3 Smart Contract Interaction

Action	Contract	Function	Modifier
Add a book	LibraryRegistry	addBook(id, title, author)	onlyOwner
Borrow a book	LibraryRegistry	borrowBook(bookId)	whenNotPaused
Return a book	LibraryRegistry	returnBook(bookId)	whenNotPaused
Register user	LibraryRegistry	registerUser(name)	—
Mint tokens	LibraryCoin	mint(to, amount)	onlyOwner
Transfer tokens	LibraryCoin	transfer(to, amount)	—
Pause system	LibraryRegistry	pause()	onlyOwner
Resume system	LibraryRegistry	resume()	onlyOwner
Transfer admin	LibraryRegistry	transferOwnership(newAdmin)	onlyOwner

Get admin	LibraryRegistry	getAdmin()	view (public)
-----------	-----------------	------------	---------------

7. Database Design (On-Chain Data Structures)

There is no traditional relational database in this system. All persistent data is stored directly in smart contract state variables on the Ethereum blockchain. The following data structures define the on-chain schema:

7.1 LibraryRegistry.sol — Core Data Structures

Book Struct

```
struct Book {
    uint256 id;           // Unique book identifier
    string  title;        // Book title
    string  author;       // Author name
    bool    isAvailable;  // True if not currently borrowed
}
```

State Variables

```
mapping(uint256 => Book)    public books;           // Book registry
mapping(address => string) public userNames;        // User display names
mapping(address => bool)    public registeredUsers; // Registration status
address public admin;       // Admin wallet address
bool    public paused;      // System pause state
uint256 public bookCount;   // Total books registered
```

Events Emitted

```
event BookAdded(uint256 indexed bookId, string title, string author);
event BookBorrowed(uint256 indexed bookId, address indexed borrower);
event BookReturned(uint256 indexed bookId, address indexed returner);
event UserRegistered(address indexed user, string name);
event OwnershipTransferred(address indexed previousOwner, address indexed newOwner);
event Paused(address admin);
event Unpaused(address admin);
```

7.2 LibraryCoin.sol — ERC-20 Token Data Structures

```
string public name      = "Library Coin";
string public symbol    = "LBRC";
uint8  public decimals  = 18;
uint256 public totalSupply;
mapping(address => uint256) public balanceOf;
mapping(address => mapping(address => uint256)) public allowance;

event Transfer(address indexed from, address indexed to, uint256 value);
event Approval(address indexed owner, address indexed spender, uint256 value);
```

7.3 Off-Chain Output Files

File	Format	Contents
------	--------	----------

balance_snapshot.csv	CSV	Account Address, LBRC Balance, ETH Balance
abi/Library.json	JSON	ABI for LibraryRegistry contract (auto-generated)
abi/LibraryCoin.json	JSON	ABI for LibraryCoin contract (auto-generated)
config/settings.py	Python	Deployed contract addresses, admin key, Ganache URL
artifacts/analytics/reports	Text	Generated history reports and snapshots

8. API Documentation

The system exposes no REST or GraphQL API. Instead, it interacts with smart contracts directly through web3.py function calls. The following documents all publicly callable contract functions, their parameters, return values, and access control.

8.1 LibraryRegistry.sol — Public Functions

Function	Parameters	Returns	Access	Description
addBook()	id: uint256, title: string, author: string	—	onlyOwner	Registers a new book in the library
borrowBook()	bookId: uint256	—	Any User + whenNotPaused	Marks a book as borrowed by caller
returnBook()	bookId: uint256	—	Any User + whenNotPaused	Marks a book as returned (available)
registerUser()	name: string	—	Any address	Saves a display name for the caller's wallet
batchAddBooks()	ids[], titles[], authors[]	—	onlyOwner	Adds multiple books in one transaction
pause()	—	—	onlyOwner	Halts all user-facing operations
resume()	—	—	onlyOwner	Restores normal operations
transferOwnership()	newAdmin: address	—	onlyOwner	Transfers admin rights to a new address
getAdmin()	—	address	Public (view)	Returns the current admin address
getBook()	bookId: uint256	Book struct	Public (view)	Returns book details by ID
getUserName()	addr: address	string	Public (view)	Returns display name for a wallet

8.2 LibraryCoin.sol — ERC-20 Functions

Function	Parameters	Returns	Access
mint()	to: address, amount: uint256	—	onlyOwner
transfer()	to: address, amount: uint256	bool	Any token holder

approve()	spender: address, amount: uint256	bool	Any token holder
transferFrom()	from, to: address, amount: uint256	bool	Approved spender
balanceOf()	account: address	uint256	Public (view)
allowance()	owner, spender: address	uint256	Public (view)

8.3 Example web3.py Interaction

```
from web3 import Web3
w3 = Web3(Web3.HTTPProvider('http://127.0.0.1:8545'))

# Call a view function
admin_addr = contract.functions.getAdmin().call()

# Send a state-changing transaction
tx = contract.functions.borrowBook(1).build_transaction({
    'from': user_address,
    'nonce': w3.eth.get_transaction_count(user_address),
    'gas': 200000,
    'gasPrice': w3.eth.gas_price
})
signed = w3.eth.account.sign_transaction(tx, private_key)
tx_hash = w3.eth.send_raw_transaction(signed.rawTransaction)
```

9. Installation & Setup Guide

9.1 Prerequisites

Requirement	Version	Install Command
Python	3.9 or higher	python.org/downloads
Node.js	16.x or higher	nodejs.org
Ganache CLI	Latest	npm install -g ganache
Git	Any	git-scm.com

9.2 Step-by-Step Setup

- **Step 1: Clone the Repository**

```
git clone https://github.com/ShahdMamdouhMuhamed/-Campus-Library-Book-Registry.git
cd -Campus-Library-Book-Registry
```

- **Step 2: Create & Activate Virtual Environment**

```
python -m venv venv
# Windows:
venv\Scripts\activate
# macOS/Linux:
source venv/bin/activate
```

- **Step 3: Install Python Dependencies**

```
pip install -r Requirements
```

- **Step 4: Start Ganache (in a new terminal)**

```
ganache --deterministic --accounts 10 --port 8545
# Copy the first account's private key shown in output
```

- **Step 5: Configure Settings**

```
# Edit config/settings.py:
GANACHE_URL = 'http://127.0.0.1:8545'
ADMIN_PRIVATE_KEY = '0x...' # First Ganache account key
ADMIN_ADDRESS = '0x...' # First Ganache account address
```

- **Step 6: Deploy Contracts**

```
python autosetup.py
# or: python scripts/deploy.py
# Contract addresses are auto-saved to config/settings.py
```

- **Step 7: Launch the Terminal Application**

```
python LibraryApp.py
```

10. Folder Structure Explanation

Path	Type	Description
LibraryApp.py	Python	Main entry point — launches the terminal application
autosetup.py	Python	Auto-deployment script: compiles, deploys, seeds books
balance_snapshot.py	Python	Root-level shortcut for balance snapshot export
balance_snapshot.csv	CSV	Output file: account balances snapshot
Requirements	Text	Python package dependencies list
api.txt	Text	API reference notes
package.json / package-lock.json	JSON	Node.js configuration (for Ganache)
· GITIGNORE	Config	Git ignore rules for secrets and build artifacts
README.md	Markdown	Project overview and quick-start guide
team_division.pdf	PDF	Team member roles and task assignment
abi/	Directory	Auto-generated ABI JSON files for both contracts
abi/Library.json	JSON	ABI for LibraryRegistry smart contract
abi/LibraryCoin.json	JSON	ABI for LibraryCoin ERC-20 contract
artifacts/	Directory	Compiled contract artifacts (Truffle-style)
artifacts/analytics/	Directory	Analytics and reporting scripts
artifacts/analytics/admin_dashboard.py	Python	Admin system summary dashboard
artifacts/analytics/history_report.py	Python	Most-borrowed books report
artifacts/analytics/ownership_transfer_test.py	Python	Ownership transfer verification
artifacts/analytics/personal_history.py	Python	Per-address activity history
artifacts/analytics/reports	Directory	Generated report outputs
artifacts/Library.sol	Solidity	Core smart contract source code

artifacts/LibraryCoin.sol	Solidity	ERC-20 token smart contract source code
artifacts/Library.json	JSON	Compiled artifact for LibraryRegistry
artifacts/LibraryCoin.json	JSON	Compiled artifact for LibraryCoin
config/	Directory	Application configuration
config/settings.py	Python	Ganache URL, deployed addresses, admin keys
scripts/	Directory	Utility and automation scripts
scripts/deploy.py	Python	Contract deployment script
scripts/balance_snapshot.py	Python	Balance snapshot with CSV export
notepad scripts/	Directory	Reference/draft script notes
notepad scripts/deploy.py	Python	Draft deployment notes
node_modules/	Directory	Node.js packages (Ganache dependencies)
__pycache__/	Directory	Python bytecode cache (auto-generated)

11. User Roles and Permissions

The system implements a two-tier role model enforced at the smart contract level using Solidity modifiers. Roles cannot be spoofed or bypassed — all enforcement happens on-chain.

Permission / Action	Admin	Normal User
Add a single book	YES	NO (reverts)
Add books in batch	YES	NO (reverts)
Mint Library Coins (LBRC)	YES	NO (reverts)
Pause / Resume the system	YES	NO (reverts)
Transfer admin ownership	YES	NO (reverts)
View admin dashboard	YES	NO
Borrow a book	YES (if unpaused)	YES (if unpaused)
Return a book	YES (if unpaused)	YES (if unpaused)
Register a display name	YES	YES
View book availability	YES	YES
Check any address balance	YES	YES
View personal activity history	YES	YES
Transfer LBRC tokens	YES	YES (if balance > 0)
View contract admin address	YES	YES (public)

11.1 How Admin Identity is Determined

Admin identity is set once in the constructor as `msg.sender` — the address that deploys the contracts. The `onlyOwner` modifier checks `require(msg.sender == admin)` before every sensitive operation. The admin address is stored in a public state variable and can be verified by anyone using `getAdmin()`.

11.2 Terminal-Side Admin Authentication

When a user selects 'Admin Login' in the terminal app, they are prompted for the admin private key (or a password-protected equivalent). The app uses this key to sign transactions as the admin address. If the key does not correspond to the admin address on-chain, all admin transactions will be reverted by the contract.

12. Workflow / Business Logic

12.1 Book Borrowing Flow

- User selects 'Borrow a Book' from the terminal menu.
- The app prompts for the Book ID.
- The app calls `borrowBook(bookId)` on `LibraryRegistry.sol`.
- The contract checks: (a) system is not paused; (b) book exists and `isAvailable == true`.
- If checks pass: `isAvailable` is set to `false`; a `BookBorrowed` event is emitted.
- The transaction hash is returned to the user as confirmation.
- The live alert monitor (if running) prints: 'ALERT: A book was just borrowed!'

12.2 Book Return Flow

- User selects 'Return a Book' from the terminal menu.
- The app prompts for the Book ID.
- The contract checks: (a) system is not paused; (b) book exists and `isAvailable == false`.
- If checks pass: `isAvailable` is set to `true`; a `BookReturned` event is emitted.
- The transaction receipt confirms the return on-chain.

12.3 Admin — Add Book Flow

- Admin selects 'Add a Book' from the admin menu.
- The app prompts for Book ID, Title, and Author.
- `addBook()` is called; the `onlyOwner` modifier verifies `msg.sender == admin`.
- If verified: the Book struct is stored in the books mapping; `BookAdded` is emitted.
- If not admin: the transaction is reverted with 'Not authorized'.

12.4 User Registration Flow

- On first launch, the terminal app checks if the user's address is registered.
- If not registered, the user is prompted to enter a display name.
- `registerUser(name)` is called, storing the name in `userNames[msg.sender]`.
- On subsequent launches, the registered name is displayed in the menu header.

12.5 Emergency Pause Flow

- Admin selects 'Pause System' from the admin menu.
- `pause()` is called on the contract; the paused state variable is set to `true`.
- All functions with the `whenNotPaused` modifier immediately begin reverting.
- Admin resumes with `resume()`, setting `paused = false`.
- All user-facing operations return to normal.

13. Screens Description (Terminal Interface)

The system uses a Python command-line interface. The following describes each screen (menu) available in the application.

Main Menu (Startup Screen)

- Displays a welcome banner with the registered user's name (if registered).
- Presents options: [1] Browse Books [2] Borrow a Book [3] Return a Book
- [4] Check Balance [5] My Activity History [6] Admin Login [0] Exit
- New users are automatically prompted to register a display name before the menu appears.

Admin Menu (After Admin Login)

- Displayed after successful admin key verification.
- [1] Add a Book [2] Batch Add Books [3] Mint Library Coins
- [4] Admin Dashboard [5] Pause System [6] Resume System
- [7] Transfer Ownership [0] Back to Main Menu

Admin Dashboard Screen

- Prints a formatted summary pulled from the blockchain:
- Total books registered, total LBRC minted, total transaction count.
- Top 3 most active user addresses with their transaction counts.

Book Browsing Screen

- Lists all registered books with their ID, title, author, and status.
- Status is shown as: [AVAILABLE] or [BORROWED].

Borrow / Return Screens

- Prompts the user for a Book ID.
- Sends the transaction and displays the transaction hash on success.
- Displays an error message if the book is already borrowed/already returned.

Personal Activity History Screen

- Prompts for an Ethereum address (defaults to the user's own).
- Scans the full blockchain event log and prints a table:
- Block Number | Action (Borrowed/Returned) | Book ID | Timestamp

Coin & ETH Balance Checker Screen

- Prompts for an Ethereum address.
- Displays: ETH Balance (in ETH) and LBRC Token Balance (in LBRC).

Balance Snapshot Screen (Admin)

- Scans all Ganache accounts.
- Exports a CSV file: Account Address, LBRC Balance, ETH Balance.
- Confirms the CSV file path on completion.

Security Test Output Screen

- Runs automated tests showing that non-admin calls to admin functions are rejected.
- Prints PASS/FAIL for each test case with the revert reason.

14. Challenges and Solutions

Gas Estimation Errors

- **Challenge:** web3.py sometimes failed to estimate gas for complex transactions, causing transactions to fail silently.
- **Solution:** Implemented explicit gas limits in all transaction calls. Added try-except blocks to catch and display detailed error messages from the contract.

Contract Recompilation on Every Run

- **Challenge:** Early versions recompiled contracts on every script execution, significantly slowing development cycles.
- **Solution:** Implemented ABI caching — after first deployment, ABIs are saved to the abi/ directory and loaded directly on subsequent runs.

Private Key Management

- **Challenge:** Hardcoding private keys in source files posed a security risk, especially when using version control.
- **Solution:** Isolated all sensitive values to config/settings.py, which is listed in .gitignore to prevent accidental commits.

Block Scanning Performance

- **Challenge:** Scanning all blocks to build activity history became slow as the number of test transactions grew on the local chain.
- **Solution:** Used event filtering (web3.py getLogs with fromBlock/toBlock parameters) instead of iterating every block manually.

Batch Transaction Atomicity

- **Challenge:** Ensuring that a batch book-add operation either fully succeeds or fully reverts required careful Solidity design.
- **Solution:** Used require() statements inside the for-loop in batchAddBooks(). Solidity's transaction atomicity ensures full rollback on any failure.

Ganache Determinism

- **Challenge:** Without deterministic mode, account addresses changed on every Ganache restart, breaking the admin address configuration.
- **Solution:** Used the --deterministic flag with Ganache CLI to ensure the same set of accounts is generated on every run.

ERC-20 Access Control

- **Challenge:** Standard ERC-20 does not include an onlyOwner modifier for mint().
- **Solution:** Extended the ERC-20 base with a custom admin state variable and modifier, giving the deployer exclusive minting rights.

15. Future Improvements

- **Web Frontend (DApp UI):** Replace the terminal interface with a React.js web application using ethers.js or web3.js, enabling a browser-based experience with MetaMask integration.
- **IPFS Integration for Book Metadata:** Store book cover images and extended metadata (ISBN, description, genre) on IPFS, referencing the content hash from the smart contract.
- **Public Testnet Deployment:** Deploy contracts to Ethereum testnets (Sepolia, Goerli) to demonstrate real-world blockchain interaction beyond the local Ganache environment.
- **NFT Book Receipts:** Issue ERC-721 NFT tokens as verifiable borrow receipts, giving users a permanent on-chain proof of their library activity.
- **Reputation & Reward System:** Automatically award LBRC tokens to users who return books on time, and implement a penalty mechanism for late returns.
- **Multi-Admin Support:** Implement a multi-signature admin model (e.g., 2-of-3) to prevent any single point of administrative failure.
- **Decentralized Search & Indexing:** Integrate The Graph protocol to index on-chain events and enable fast, decentralized queries for book availability and user history.
- **Mobile Application:** Develop a mobile app using React Native with WalletConnect integration for smartphone-based library access.

16. Testing Strategy

The project uses a combination of automated Python unittest tests and manual end-to-end testing against the local Ganache blockchain.

16.1 Security Tests (security_test.py)

These automated tests verify that access control rules are correctly enforced at the smart contract level.

Test ID	Action	Expected Result	Status
TC-001	Normal user calls addBook()	Transaction reverts with 'Not authorized'	PASS
TC-002	Normal user calls mint()	Transaction reverts	PASS
TC-003	Normal user calls pause()	Transaction reverts	PASS
TC-004	Normal user calls transferOwnership()	Transaction reverts	PASS
TC-005	Admin calls addBook()	Book added successfully	PASS
TC-006	Borrow while paused	Transaction reverts with 'System paused'	PASS
TC-007	Return while paused	Transaction reverts with 'System paused'	PASS
TC-008	Borrow unavailable book	Transaction reverts	PASS

16.2 Ownership Transfer Tests (ownership_transfer_test.py)

- **Admin performs an action before transfer:** Action succeeds
- **transferOwnership(newAdmin) is called:** OwnershipTransferred event emitted
- **Old admin attempts admin action:** Transaction reverts
- **New admin performs admin action:** Action succeeds

16.3 Manual End-to-End Tests

- Deploy contracts → verify addresses saved to config/settings.py
- Register user → verify name appears in terminal header
- Add book → verify book appears in browse list with AVAILABLE status
- Borrow book → verify status changes to BORROWED
- Return book → verify status changes back to AVAILABLE
- Mint LBRC → verify balance_checker shows updated token balance
- Balance snapshot → verify CSV contains correct addresses and balances

- History report → verify most-borrowed books table is accurate

17. Deployment Process

17.1 Local Deployment (Development)

The standard deployment process for local development and testing:

- **Start Ganache:**

```
ganache --deterministic --accounts 10 --port 8545
```

- **Set Admin Configuration:**

```
# In config/settings.py:  
ADMIN_PRIVATE_KEY = ''  
ADMIN_ADDRESS = ''
```

- **Run Auto-Setup:**

```
python autosetup.py  
# This script:  
# 1. Compiles LibraryRegistry.sol and LibraryCoin.sol using solcx  
# 2. Deploys both contracts to Ganache  
# 3. Saves deployed addresses to config/settings.py  
# 4. Saves ABIs to abi/ directory  
# 5. Seeds 3 sample books to the registry
```

- **Verify Deployment:**

```
python LibraryApp.py  
# Select 'Browse Books' to confirm seeded books are on-chain
```

- **Run Security Tests:**

```
python artifacts/analytics/ownership_transfer_test.py
```

17.2 Contract Compilation Details

Contracts are compiled using py-solc-x (a Python wrapper for the Solidity compiler). The deploy script automatically installs the required Solidity compiler version if not already present:

```
import solcx  
solcx.install_solc('0.8.0')  
solcx.set_solc_version('0.8.0')  
  
compiled = solcx.compile_source(  
    contract_source_code,  
    output_values=['abi', 'bin']  
)
```

17.3 Testnet Deployment (Future)

For public testnet deployment (e.g., Ethereum Sepolia testnet), the process would be:

- Replace GANACHE_URL with a public RPC endpoint (e.g., Infura, Alchemy)
- Fund the admin address with testnet ETH via a faucet
- Run the same deployment scripts — they are network-agnostic

- Save deployed contract addresses for frontend or API integration
- Verify contracts on Etherscan for public transparency

18. Conclusion

The Campus Library Book Registry successfully demonstrates how blockchain technology can be applied to solve real-world data integrity and transparency challenges in an institutional setting. By replacing a mutable, centralized database with an immutable Ethereum smart contract, the system ensures that every library transaction is permanently recorded, independently verifiable, and impossible to tamper with after the fact.

The project encompasses the full stack of blockchain application development: Solidity smart contract authoring with role-based access control, ERC-20 token deployment, Python-based web3 integration, terminal application design, on-chain analytics, automated security testing, and comprehensive documentation.

Key achievements of this project include:

- Two fully functional Solidity smart contracts deployed on a local Ethereum network
- A complete ERC-20 token (Library Coin / LBRC) with admin-restricted minting
- Role-based access control enforced at the contract level using Solidity modifiers
- An emergency pause/resume mechanism for system safety
- A Python terminal application supporting full Admin and User workflows
- Automated security tests verifying that access control rules cannot be bypassed
- On-chain analytics: admin dashboard, activity history, and balance snapshots
- A live alert monitoring system for real-time blockchain event tracking
- Complete technical documentation covering all aspects of the system

This project serves as a strong foundation for future development toward a production-grade decentralized library system, with clear pathways to add a web frontend, IPFS integration, NFT-based receipts, and public testnet or mainnet deployment.

Development Team

Member	Role	Key Responsibilities
Shahd Mamdouh	Lead / Blockchain Engineer	Core smart contract, access control, batch operations, pause/resume, ownership transfer
Menna Sherif	Coin & Deployment Engineer	ERC-20 Library Coin, deploy scripts, user registration (Solidity side), balance CSV, README
Moustafa Said	Terminal Application Developer	Terminal app design, transaction sender, user registration (UI), balance checker
Mustafa Waheed	Data & Analytics Engineer	Admin dashboard, history report, activity history, ownership transfer tests
Marwan Ashraf	Security & QA Engineer	Security tests, live alert monitor, integration testing, demo preparation

Technologies of Cryptocurrencies — Final Project
Campus Library Book Registry v1.0 | May 2026